GitHub 週間スター上位レポート

2026-10-30 作成 ／ 対象: 2026-10-23 以降に作成されたリポジトリ
テーマ: WebLLM

#	区分	リポジトリ	★	一言で
1	テーマ	theletterf/living-house	2	ブラウザ内で動く小型LLMが、利用者の操作を観察してWebアプリのUIをリアルタイムに作り変える実験的プロジェクト
2	テーマ	soumwadeep/ghostlight	1	Gemma 2をブラウザ内で動かし、故人の思い出からオフレンダと追悼文を作るローカルファーストのWebアプリ
3	テーマ	dadashi44/vue-llm	1	Vue と Nuxt 向けに、ストリーミング対応の useChat やツール呼び出し、ブラウザ内 LLM 実行を提供する AI SDK
4	テーマ	lf735/webllm	0	Gemma 4系モデルをブラウザ内(WebGPU)またはOllama経由でローカル実行するWebチャットUI
5	テーマ	Kita12123/webllm-poc	0	ブラウザ内でLLMを動かすWebLLM（@mlc-ai/web-llm）を試すための、TypeScript製の個人検証用ミニマムデモ
6	総合	unclebob/ub-mode	5,056	AIコーディングエージェントに規律あるソフトウェア工学の原則を守らせるためのスキル定義ファイル（SKILL.md）
7	総合	rddmwmddr/android-ad-cleaner	4,902	中国メーカー製Androidスマホのシステム広告・ホット検索・ニュース推薦・営銷通知を、root不要かつ元に戻せる形でオフにするツール群
8	総合	JustAResearcher/gpu-unlock-ada	4,546	RTX 4090などAda世代GPUのFP32累積テンソル演算性能を約2倍に引き上げるLinuxカーネルモジュール
9	総合	interplains/Zero-Breach-Toolkit	3,701	Pegasus や Predator など既知の傭兵型スパイウェアの痕跡を、PC やスマホのデータからオフラインで調べる無料のPython製ツール
10	総合	anhao/ai-pose-gallery-zh	3,612	日本語のAIポーズ参考画像サイトを全量ミラーし、中国語化した上で依存ゼロの静的ギャラリーとして公開するリポジトリ

テーマ 1. theletterf/living-house

https://github.com/theletterf/living-house

★ 2 ／ フォーク 0 ／ 言語 JavaScript ／ 作成 2026-10-23 ／ MIT
該当テーマ: WebLLM

ブラウザ内で動く小型LLMが、利用者の操作を観察してWebアプリのUIをリアルタイムに作り変える実験的プロジェクト

概要

Living Houseは、WebGPU上のWebLLMで小型言語モデルをブラウザ内だけで動かし、ユーザーのクリックやためらい、テキスト選択などの操作シグナルをもとに、UIの文言、CSS、ヒント、並び順、ピン留めを動的に変更する適応型UIの実験です。題材は、照明やブラインド、鍵などを扱うスマートホームアプリ「Hearth」と、あえて分かりにくいラベルを付けたワークスペースアプリ「Atrium」です。要素IDを不変に保ちつつ見た目や名前だけを変える設計により、レイアウトが人ごとに異なっても追従するライブヘルプを実現している点が新しいところです。推論・ログ・計測はすべてローカルに留まり、ビルド不要の静的ファイルとして動きます。

要点

- WebLLMでQwen2.5/Qwen3/Llama 3.2をブラウザ内実行
- 操作シグナルから文言・スタイル・ヒント・順序・ピンを変更
- JSONスキーマ制約付きデコードと事前検証で安全に適用
- 不変IDに基づくテンプレートでレイアウトに追従するヘルプ
- ビルド不要、静的サーバーで起動しテレメトリなし

何をするものか

Living Houseは「小型モデルが、使っている人に合わせてインターフェースをリアルタイムに作り変えられるか」を問う適応型UIの実験です。Simon Edwardsson氏のWorldTUI（小型モデルが端末画面の変化を予測する試み）に着想を得ており、名前はRemedyのゲーム『Control』に登場する、歩く人に合わせて自らを組み替える建物Oldest Houseにちなんでいます。対象アプリはスマートホームアプリ「Hearth」と、請求書がAccount → Statementsに隠れているなど、わざと曖昧なラベルを持つワークスペースアプリ「Atrium」の2つです。Atriumは、モデルが分かりにくい文言を直せるかを試す用途に向いています。

主な機能

モデルが変更できるのは5種類です。テキストの書き換え、24種類のCSSプロパティから1つを選ぶスタイル変更、コントロール下への一行ヒント、ツールバー先頭への移動、Quick actionsバーへのピン留めです。ユーザーは要素を右クリックして「Change this」に指示を書くか、⌘K/Ctrl+Kのask barで全体への要望を出せます。「Rename Overview to Home」のような明確なコマンドはモデル推論なしで実行されます。適応の強さはGentle、Balanced、Max、Extremeの4段階から選べます。そのほか、変更ごとの承認ポップオーバー、Undo、セッション終了時のサマリー、レイアウトの保存、JSONLログのエクスポートも備えています。

仕組み・アーキテクチャ

推論は専用のWeb Worker（model-worker.js）で行い、UIスレッドと分離しています。毎ラウンド、要素一覧（テキスト、目的、状態、カスタムスタイル、利用統計）、直近の操作、ユーザーの継続的な要望をモデルに渡し、JSONで回答させます。デコードは変更タイプごとに形の異なるJSONスキーマで制約され、idも適用可能な要素の種類に限定されます。スキーマはロック状態に依存しないため、WebLLMはコンパイル済み文法をラウンド間で再利用できます。提案はCSS.supportsで検証され、url()や@importなど外部を読み込む値は拒否されます。アプリ部分はCSS containmentで囲われているため、UndoやStart overは常に操作できます。

技術構成

依存パッケージやビルドステップはなく、素のJavaScript、HTML、CSSで構成されています。モデルランタイムのWebLLMはCDNから読み込みます。ファイルの役割は次のとおりです。scenarios.jsはアプリ定義をデータとして持ち、docs.jsはIDベースのヘルプ記事を収めています。app.jsは要素レジストリ、描画、シグナル記録、検証、Undoを担当します。adapters.jsはモデル読み込み、プロンプト構築、スキーマ制約デコードを、dom.jsはフォーカスを保った差分DOM更新を、requests.jsは直接コマンドの解析を、performance.jsはローカルの性能集計を担います。テストはNode.jsの組み込みテストランナーで書かれています。

使い方・導入方法

WebGPU対応の最近のChromeまたはEdgeが必要です。リポジトリをcloneし、python3 -m http.server 8642 などの静的サーバーで配信して http://localhost:8642 を開きます。続いてQwen2.5（0.5B/1.5B/3B）、Qwen3（1.7B/4B、thinkingオフ）、Llama 3.2 3Bからモデルを選びます。初回は数百MB〜数GBのダウンロードが発生しますが、以降はブラウザにキャッシュされます。READMEでは、Qwen2.5 1.5Bが実用上の最小モデルとされ、M系ノートPCで1ラウンド約4〜6秒とあります。プロンプト、許可する変更、観測するシグナル、閾値、温度はセットアップ画面で編集できます。node --test *.test.mjs を実行すれば、モデルをダウンロードせずにロジックを検証できます。

注目ポイント・注意点

最も興味深いのは、人ごとに形が変わるUIをどう文書化するかへの回答です。ヘルプ記事は{{nav_overview}}のような要素IDで書かれ、現在の名前、旧名、ライブ状態、ピン状態に応じて描画されます。各名前のShow meリンクからは、該当要素をハイライト表示できます。提案が拒否され続けたり操作上の摩擦がなくなったりすると、セッション終了を提案する検出機能もあります。エクスポートしたログは、将来的に役立った変更で小型モデルを学習させる用途を想定しています。注意点として、0.5Bモデルは自動リネームができません。また、汎用的な要望は誤解されることがあるため、Undoでの確認が前提です。テーマはライトのみで、スター数も少ない初期段階の実験的リポジトリです。

技術構成: JavaScript、HTML、CSS、WebLLM、WebGPU、Web Worker、Qwen2.5 / Qwen3 / Llama 3.2、Node.js test runner
向いている人: 適応型UIやブラウザ内LLM（WebLLM/WebGPU）の活用に関心があるフロントエンドエンジニア、UXや人間とAIのインタラクションを研究する学生・研究者に向いています。小型モデルの構造化出力をJSONスキーマで制約し、安全にDOMへ反映する設計の参考にもなります。

テーマ 2. soumwadeep/ghostlight

https://github.com/soumwadeep/ghostlight

★ 1 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-10-27 ／ MIT
該当テーマ: WebLLM

Gemma 2をブラウザ内で動かし、故人の思い出からオフレンダと追悼文を作るローカルファーストのWebアプリ

概要

Ghostlightは、死者の日（Día de Muertos）をモチーフに、大切な人についての6つの問いへの回答から追悼文、インタラクティブなオフレンダ（祭壇）、オフラインで開ける記念HTMLを生成するアプリです。Gemma 2（2B）をWebLLMとWebGPUでブラウザのタブ内で動かし、記録はIndexedDBに保存されるため、バックエンドやアカウント、クラウド推論は一切ありません。質問への回答はまずアーカイブから検索し、該当がなければ回答を拒否し、引用はモデルではなくアプリ側が付与するという、幻覚を抑える設計が特徴です。Hacktoberfest 2026 DEV Challenge「Ghost in the Machine」向けに作られました。

要点

- Gemma 2 2BをWebLLM+WebGPUでブラウザ内実行
- バックエンド・APIキー・環境変数なしの静的サイト
- 検索優先の回答と拒否で幻覚を抑える設計
- モデル不要のcandle modeとOllama連携も選択可
- PWA対応で初回DL後は完全オフライン動作

何をするものか

Ghostlightは、思い出の人について「料理」「歌」「場所」「口癖」「小さな物」「やり残したこと」という6つの問いを投げかけ、その回答から三つの成果物を作ります。一つ目は回答の言葉を元にした短い追悼文、二つ目は6本のろうそく、マリーゴールド、水、パペルピカド、回答に含まれる供物を配した段状のオフレンダで、ろうそくをクリックすると元の思い出が読めます。三つ目はスクリプトを含まない単一のHTMLファイルとしての記念品で、アプリなしでオフラインで開けます。READMEは、これは霊媒でもセラピーでもなく、言葉を置く場所だと明言しています。

主な機能

ブラウザ内のGemma 2による追悼文の執筆、アーカイブに基づいて質問に答えるAskパネル、複数のオフレンダを管理できるローカルアーカイブ（JSONのインポート・エクスポートとワンクリック削除）、記念HTMLのエクスポートを備えます。モデルをダウンロードしないcandle modeでは、ユーザー自身の文章を並べ替えるだけで書き換えません。ローカルのOllamaを使うオプションもあり、接続できない場合はクラウドではなくメモリ検索にフォールバックします。ブラウザの音声APIは音声を外部に送ることがあるため、意図的にマイク入力を持たず入力はタイピングのみです。キーボード操作、フォーカス表示、reduced-motion対応などアクセシビリティにも配慮しています。

仕組み・アーキテクチャ

処理の流れは、6つの回答をIndexedDBのアーカイブに保存し、メモリ検索を経てGemma 2で150〜200語・3段落の追悼文を生成し、オフレンダと記念品を作るというものです。AIの仕事は二つに分かれます。Weaveは記憶からのみ三人称で書き、各段落を特定の記憶に割り当てます。Answerはまず記憶を検索し、一致がなければ「The flame has no memory of that.」と拒否、一致した場合もモデルは言い換えるだけで、引用はアプリが付与します。プロンプトはsrc/lib/ai/prompts.tsにあり、About画面にそのまま表示されます。src/lib/aiにはretrieval、candle、weave、ask、webllm、ollama、parseなどが分かれて実装されています。

技術構成

React 19、Vite 8、TypeScript 6を基盤に、ルーティングにreact-router-dom、アニメーションにmotion、アイコンにlucide-react、IndexedDB操作にidb-keyval、ブラウザ内LLM実行に@mlc-ai/web-llm、PWA化にvite-plugin-pwaを使っています。フォントはFraunces、Instrument Sans、IBM Plex Mono。テストはvitest、リンターはoxlint、devDependenciesにplaywrightも含まれます。モデルはgemma-2-2b-it-q4f16_1-MLC（約1.4GB）で、Hugging Faceのmlc-aiリポジトリから一度だけ取得されます。READMEによると、WebLLM 0.2.86ではGemma 3 1Bのビルドが読み込めない上流の問題があるためGemma 2を採用しており、モデルIDはsrc/lib/ai/webllm.tsの1行で差し替え可能とされています。

使い方・導入方法

git cloneの後、npm installとnpm run devを実行し、http://localhost:5173 を開くだけで動きます。環境変数、APIキー、外部サービスの設定は不要です（Node.jsは22以上）。weave画面で「Summon Gemma 2」を選ぶとWebGPU経由でモデルを使え、デスクトップのChromeかEdge、約2GBの空きグラフィックメモリが推奨です。テスト環境では1.4GBのダウンロードに約2分、追悼文生成に13.6秒だったと記載されています。npm testで20個のユニットテスト、npm run typecheck、npm run buildが利用可能。ビルドはdist/の静的サイトで、render.yamlによるRenderへのワンクリックデプロイにも対応しています。Ollamaを使う場合はOLLAMA_ORIGINS=* ollama serveで起動します。

注目ポイント・注意点

プライバシーの約束を表形式で明示し、端末外に出るのは静的ページのリクエストとモデルのダウンロードのみで、回答・写真・質問・追悼文は端末から出ないと説明しています。一方で共有PCやブラウザ拡張からは守れないこと、サイトデータを消すとアーカイブも消えることも正直に書かれています。2Bモデルは小さく、アーカイブにない細部を書くことがあるため、記憶を常に表示しcandle modeを用意しているとのことです。デモのRuma Dasは架空の人物で、デモの完成追悼文はモデル出力ではなく手書きの目標文です。WebGPU非対応環境ではモデルは動かずcandle modeのみとなります。スター数は1で、作成直後のハッカソン作品である点も留意が必要です。

技術構成: TypeScript、React 19、Vite 8、WebGPU、@mlc-ai/web-llm、Gemma 2 2B IT、IndexedDB、idb-keyval、vite-plugin-pwa、react-router-dom、motion、lucide-react、Vitest、oxlint、Playwright、Ollama（任意）、Render
向いている人: ブラウザ内LLM（WebLLM/WebGPU）やローカルファースト・オフラインファーストなPWAの実装例

を学びたいフロントエンドエンジニアや学生、検索優先・拒否付きのグラウンディング設計など小型モデルの幻覚対策に関心がある人、プライバシーを重視したAIアプリやハッカソン作品の参考を探している人に向いています。

テーマ 3. dadashi44/vue-llm

https://github.com/dadashi44/vue-llm

★ 1 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-10-24 ／ MIT
該当テーマ: WebLLM

Vue と Nuxt 向けに、ストリーミング対応の useChat やツール呼び出し、ブラウザ内 LLM 実行を提供する AI SDK

概要

vue-llm は、Vue 3.3 以上と Nuxt で LLM チャット機能を組み込むための TypeScript 製 SDK です。リアクティブな useChat() を中心に、OpenAI、Anthropic、Gemini、Ollama を同じインターフェースで扱えます。WebGPU を使ってブラウザ内でモデルを動かす機能、ツール呼び出しを Vue コンポーネントとして描画する Generative UI、Vue DevTools 連携も備えています。ランタイム依存は peer dependency の vue だけで、Vue 2.7 でも composable を利用できます。

要点

- useChat() でストリーミング、中断、再生成を数行で実装できる
- OpenAI、Anthropic、Gemini、Ollama、WebGPU を同じ API で扱える
- ツール呼び出しを Vue コンポーネントとして描画する Generative UI を備える
- リトライ、レート制限、永続化、プロンプトテンプレートを標準で提供
- Nuxt モジュールと DevTools タブがあり、ランタイム依存は vue のみ

何をするものか

vue-llm は、Vue と Nuxt のアプリに LLM を使ったチャットや生成機能を組み込むためのライブラリです。README では「The AI SDK for Vue and Nuxt」と説明されています。中心となるのは useChat() という composable で、messages、input、handleSubmit、isLoading などのリアクティブな値と関数を返します。README によれば、ストリーミング、ツール呼び出し、マルチプロバイダー対応のチャットを約 10 行で書けます。ほかに useCompletion と useObject もあり、ドキュメントの構成を見るとテキスト補完や構造化出力にも使えるようです。公式ドキュメントのサイトには、API キーなしで試せるプレイグラウンドが用意されています。

主な機能

プロバイダーは createOpenAI、createAnthropic、createOllama、Google 向けの実装、それに WebGPU で動く createWebLLM が用意されています。ツール呼び出しは tools 定義と onToolCall で扱い、maxToolRoundtrips で往復回数の上限を設定できます。添付ファイルは画像、PDF、音声、テキストに対応します。プロバイダーごとの対応範囲は表にまとめられており、対応外の組み合わせはリクエスト送信前にエラーになります。永続化は createWebStorage（localStorage）、createMemoryStorage、または ChatStorage を実装した独自バックエンドで行います。さらに、型推論付きのプロンプトテンプレート（createPrompt、createChatPrompt、usePrompt）と、shadcn-vue 経由で追加できるチャット UI も提供されています。

仕組み・アーキテクチャ

ソースコードは core、composables、providers、components に分かれています。core には sse-parser、stream-processor、partial-json、markdown、storage などが置かれています。メッセージは parts ベースの UIMessage 形式で、text、reasoning、tool-call、file、source といったパートを持ちます。tool-call パートは streaming、pending、running、result、error の状態を持ち、MessageParts コンポーネントで独自の Vue コンポーネントとして描画できます。プロバイダーには wrapProvider でミドルウェアを重ねられ、retryMiddleware（指数バックオフ、Retry-After 対応）や rateLimitMiddleware を適用できます。サーバー側では vue-llm/server の createLLMHandler でツール実行やクライアント単位のレート制限ができ、createSSEProvider で独自の SSE API にも接続できます。

技術構成

言語は TypeScript で、ビルドには Vite と vue-tsc を使っています。パッケージは ESM と CJS の両方で配布され、エクスポートは vue-llm、vue-llm/providers、vue-llm/components、vue-llm/server、vue-llm/devtools、vue-llm/nuxt に分かれています。peer dependency は vue（^2.7.0 または ^3.3.0）です。@mlc-ai/web-llm、@nuxt/kit、@vue/devtools-api は任意の peer dependency になっています。テストは Vitest と @vue/test-utils、jsdom を使い、Vue 2 向けの設定ファイル vitest.vue2.config.ts もあります。ドキュメントは VitePress で書かれ、英語、日本語、ロシア語、中国語、ポルトガル語、スペイン語に翻訳されています。Node.js 18 以上が必要です。

使い方・導入方法

npm install vue-llm でインストールし、vue-llm/providers から createOpenAI などでプロバイダーを作ります。それを useChat に provider と model と一緒に渡せば使えます。Nuxt では nuxt.config.ts の modules に vue-llm/nuxt を追加し、runtimeConfig.vueLlm に openaiApiKey などを設定します。こうすると composable とプロバイダーのファクトリが自動インポートされます。ブラウザ内で推論する場合は @mlc-ai/web-llm を別途インストールし、CreateMLCEngine を createWebLLM に渡します。そのうえで Llama-3.2-1B-Instruct-q4f16_1-MLC のようなモデル ID を指定します。チャット UI は npx shadcn-vue add https://vue-llm.js.org/r/chat.json で自分のプロジェクトに追加できます。

注目ポイント・注意点

セキュリティ面では、ブラウザでプロバイダーを使うと既定でエラーになり、dangerouslyAllowBrowser: true を明示した場合だけ動きます。README はこれをプロトタイプ用途に限定しており、本番ではサーバー経由のプロキシか Nuxt の runtimeConfig を使うよう勧めています。添付ファイルは data URL として保存されるため、localStorage の約 5MB の上限を超えることがあります。サーバー側のレート制限はインスタンスごとのメモリ上で動くため、複数インスタンス構成では Redis などの共有リミッターが別途必要です。retryMiddleware が再送するのは最初のリクエストだけで、ストリーム開始後のエラーは再試行されません。なお、スター数はまだ少なく、実運用での採用実績は README からは不明です。

技術構成: TypeScript、Vue 3 / Vue 2.7、Nuxt (@nuxt/kit)、Vite、Vitest、@vue/test-utils、@mlc-ai/web-llm (WebGPU)、@vue/devtools-api、VitePress、Tailwind CSS、shadcn-vue、ESLint
向いている人: Vue や Nuxt で ChatGPT 風のチャットボットや AI 機能を作りたいフロントエンドエンジニアに向いています。複数の LLM プロバイダーを切り替えたい人、ツール呼び出しの結果を独自 UI で見せたい人にも適してい ま

す。サーバーを用意せずに WebGPU でブラウザ内推論を試したい学生や開発者にも使えます。Vercel AI SDK のような体験を Vue 環境で求める人にも合います。

テーマ 4. lf735/webllm

https://github.com/lf735/webllm

★ 0 ／ フォーク 0 ／ 言語 JavaScript ／ 作成 2026-10-26
該当テーマ: WebLLM

Gemma 4系モデルをブラウザ内(WebGPU)またはOllama経由でローカル実行するWebチャットUI

概要

Gemma 4 Studioは、GoogleのGemma 4ファミリー(E2B、E4B、12B、26B MoEなど)を、クラウドにデータを送らずローカル環境で動かすためのWebインターフェースです。軽量モデルは@mlc-ai/web-llmを使ってブラウザ内のWebGPUで完結させ、大きなモデルはlocalhost:11434のOllama(またはLM Studio)に接続して実行する、2つのエンジンを切り替えられます。Thinking Modeの思考過程の可視化、画像入力、tok/sやTTFTのリアルタイム表示など、ローカルLLMの挙動を観察しやすい機能をそろえています。プライバシーを重視しつつ手軽にGemmaを試したい人向けの、Vite製の小規模フロントエンドです。

要点

- WebGPU内蔵実行とOllama接続の2エンジンを選択可能
- Gemma 4の<thought>タグを折りたたみ表示で可視化
- tok/s・TTFT・トークン数をHUDでリアルタイム表示
- 会話履歴はlocalStorageに保存、Markdown出力も可
- npm install と npm run dev だけで起動できる構成

何をするものか

Gemma 4 Studioは、Google Gemma 4系のモデルを自分のマシン上で動かし、チャット形式で対話するためのWebアプリです。README はフランス語で書かれており、データを一切クラウドに送らない「ローカルかつプライベートなAI」をうたっています。対象モデルとしてEdge向けのE2B、E4B、12B、26B MoE、31B、さらにGemma 2 2Bが挙げられています。ブラウザだけで完結する軽量モデル実行と、ローカルサーバーに処理を任せる大規模モデル実行の両方を一つのUIから扱える点が特徴です。なお説明文やライセンスはリポジトリ上で明記されておらず、package.json には ISC と記載されていますが正式なライセンスファイルは見当たりません。

2つの実行エンジン

1つ目は WebGPU (In-Browser) モードで、@mlc-ai/web-llm を用いて Gemma 4 E2B や Gemma 2 2B のような超軽量モデルを100%ブラウザ内で実行します。Chrome、Edge、Firefox Nightly など WebGPU 対応ブラウザが必要です。2つ目は Ollama モードで、localhost:11434 の REST API とストリーミングを利用し、E4B、12B、26B MoE、31B といった大きめのモデルを CUDA、Metal、Vulkan によるアクセラレーションで動かします。README では LM Studio も選択肢として挙げられていますが、LM Studio との具体的な接続方法は README からは不明です。

主な機能

Thinking Mode ビジュアライザーは、Gemma 4 が出力する <thought> タグをストリーミング中に検出し、タイマーと思考トークン数付きのアニメーション付きアコーディオンで表示します。マルチモーダル対応として、画像のドラッグ&ドロップや添付による視覚解析が可能です。HUD では tok/s、最初のトークンまでの時間(TTFT、ミリ秒)、総トークン数を表示します。そのほか、localStorage への会話自動保存と複数セッション管理、ワンクリック削除、思考過程を含む .md 形式でのエクスポート、コピーボタン付きコードブロック、Temperature・Top-P・Max Tokens の調整、Codeur・Raisonneur・Synthétique・Polyvalent といったシステムプロンプトのプリセットを備えています。

仕組み・技術構成

ファイル構成は index.html、style.css、src/main.js、vite.config.js というシンプルなもので、フレームワークを使わない素の JavaScript で書かれていると見られます。ビルドツールは Vite(devDependencies で ^8.3.5)、ランタイム依存は @mlc-ai/web-llm(^0.2.86)のみです。開発サーバーは vite --host 0.0.0.0 --port 5173 で起動するため、LAN 内の他端末からもアクセスできる設定になっています。Ollama との通信はブラウザから直接 HTTP API を呼ぶ構成のため、Ollama 側で CORS を許可する必要があります。Markdown のレンダリングやシンタックスハイライトにどのライブラリを使っているかは、依存関係に記載がなく README からは不明です。

使い方・導入方法

リポジトリのフォルダで npm install と npm run dev を実行すると、http://localhost:5173 で UI が開きます。WebGPU モードならこれだけで試せます。Ollama を使う場合は、OLLAMA_ORIGINS="*" ollama serve のように CORS を許可した状態でサーバーを起動し、ollama run gemma4:latest などでモデルを取得します。VRAM に応じて gemma4:e2b、gemma4:e4b、gemma4:12b、gemma2:2b といったバリアントも選べます。UI で Ollama タブを選ぶとインストール済みモデルが自動検出され、設定メニュー(⚙️)から接続テストも行えます。

注目ポイント・注意点

ブラウザ内推論とローカルサーバー推論を同じUIで比較でき、TTFT や tok/s を確認できるため、ローカルLLMの性能検証や学習用途に便利です。一方で、スター・フォークともに0の個人リポジトリで、ライセンスの正式表記も不明なため、再利用時は注意が必要です。OLLAMA_ORIGINS="*" はすべてのオリジンからのアクセスを許可する設定であり、開発サーバーも 0.0.0.0 で公開されるため、共有ネットワーク上では公開範囲に気を付けるべきです。また、README に記載された Gemma 4 の各モデル名が web-llm や Ollama 側で実際に提供されているかどうかは、README からは確認できません。

技術構成: JavaScript、Vite、@mlc-ai/web-llm、WebGPU、Ollama REST API、HTML/CSS、localStorage
向いている人: クラウドにデータを送らずに Gemma 系モデルを試したい開発者や学生、WebGPU によるブラウザ内LLM推論に興味がある人、Ollama 用の軽量なチャットUIを探している人に向いています。tok/s や TTFT の可視化があるため、ローカル環境でのモデル性能比較や、Thinking Mode の挙動観察といった検証・学習用途にも適しています。

テーマ 5. Kita12123/webllm-poc

https://github.com/Kita12123/webllm-poc

★ 0 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-10-25
該当テーマ: WebLLM

ブラウザ内でLLMを動かすWebLLM（@mlc-ai/web-llm）を試すための、TypeScript製の個人検証用ミニマムデモ

概要

このリポジトリは、ブラウザ上でLLMを実行するライブラリ @mlc-ai/web-llm を、Webアプリの形で動かしてみるための検証プロジェクトです。README のタイトルは「WebLLM Conversation Orchestrator App」で、WebLLM API の最小限のデモと説明されています。src に agent.ts と engine.ts があることから、エンジン部分とエージェント（会話制御）部分を分けた構成と考えられますが、具体的な実装内容は README からは不明です。Parcel でビルド・起動する手軽な構成なので、WebLLM を触り始める際の参考になります。

要点

- @mlc-ai/web-llm を使ったブラウザ内LLMの検証用デモ
- npm install と npm start だけで起動できる
- Parcel 2 でバンドルし、開発サーバーはポート8888を使用
- agent.ts と engine.ts に役割を分けたファイル構成
- スター0の個人PoCで、ライセンスは不明

何をするものか

WebLLM の検証プロジェクトです。README では「WebLLM Conversation Orchestrator App」と名付けられ、WebLLM API を Webアプリの環境で動かす最小限のデモと説明されています。WebLLM は MLC AI が提供するライブラリで、サーバーを用意せずブラウザ上で言語モデルを動かすためのものです。名前の Conversation Orchestrator から、会話の流れを制御する仕組みを試していると読み取れます。ただし、どのモデルを使い、どんな対話をするのかといった具体的な内容は README からは不明です。

仕組み・アーキテクチャ

ソースはすべて src ディレクトリにあり、index.html、index.css、index.ts、engine.ts、agent.ts の5ファイルで構成されています。ファイル名から考えると、index.html と index.css が画面、index.ts がエントリーポイントです。engine.ts は WebLLM エンジンの初期化や推論呼び出し、agent.ts は会話を進めるエージェント的なロジックを担当していると推測されます。ただし、これはあくまでファイル名からの推測であり、実際の処理内容やモジュール間の関係は README からは不明です。

技術構成

主要言語は TypeScript（devDependencies では ^4.9.5）です。ランタイムの依存は @mlc-ai/web-llm ^0.2.85 のみという、非常にシンプルな構成です。バンドラには Parcel ^2.8.3 を使っています。buffer、

process、url といった Node.js 向けモジュールのブラウザ用ポリフィルも devDependencies に入っています。tslib も含まれています。package.json の name が get-started になっていることから、WebLLM 公式の入門サンプルを元にしている可能性があります。

使い方・導入方法

README に従い、リポジトリのフォルダで npm install を実行してから npm start を実行します。npm start は「parcel src/index.html --port 8888」を呼び出すので、ブラウザで localhost:8888 を開くとアプリを確認できると考えられます。本番用のビルドは npm run build で行い、成果物は lib ディレクトリに出力されます。なお、WebLLM は WebGPU を前提としたライブラリです。そのため、対応ブラウザや GPU 環境が必要になる点に注意してください。具体的な動作要件は README からは不明です。

注目ポイント・注意点

依存関係が最小限なので、WebLLM をブラウザアプリに組み込む流れをつかむ教材として扱いやすい点が魅力です。エンジンとエージェントを分けた構成は、会話制御を自分で拡張したいときの出発点になり得ます。一方で、README はセットアップ手順だけの簡素な内容です。機能説明、対応モデル、スクリーンショットなどはありません。ライセンスも明記されていないため、コードを流用する場合は作者に確認するのが無難です。スター・フォークともに0の個人用PoCである点も踏まえて参照してください。

技術構成: TypeScript、@mlc-ai/web-llm、Parcel、HTML、CSS、Node.js / npm
向いている人: ブラウザ内でLLMを動かす WebLLM を初めて試したいフロントエンドエンジニアや学生に向いています。TypeScript と Parcel で作られた最小構成のサンプルを見ながら、会話エージェント的な仕組みを検証・拡張したい人の参考になります。

総合 6. unclebob/ub-mode

https://github.com/unclebob/ub-mode

★ 5,056 ／ フォーク 281 ／ 言語 不明 ／ 作成 2026-10-25

AIコーディングエージェントに規律あるソフトウェア工学の原則を守らせるためのスキル定義ファイル（SKILL.md）

概要

ub-mode は、AIエージェントがコードを書いたり変更したりする際に、TDD やリファクタリングなどの規律あるエンジニアリング原則を強制するためのエージェントスキルです。@poteto 氏のルール（poteto-mode / pstack）の一部をベースに内容を絞り込み、Crapper・Dryer・Mutator といった決定的な品質ゲートツールと組み合わせている点が特徴です。「まず動かす」「次に正しくする」「実践」の三段階で原則が整理されており、Claude や Cursor、Grok などのエージェントのスキルディレクトリに置くだけで使えます。

要点

- AIエージェント向けのスキル定義で、実体は SKILL.md の1ファイル
- First Make it Work / Then Make it Right / Practice の三段構成
- CRAP < 10、重複度 < 0.8 など数値基準で品質を判定
- Crapper・Dryer・Mutator など姉妹ツールとの併用が前提
- スキルディレクトリにコピーして /ub-mode で起動

何をするものか

ub-mode は、AIコーディングエージェントにソフトウェア開発の規律を守らせるための「スキル」です。エージェントがコードを書く・変更する場面で、思いつきの実装や場当たり的な修正をさせず、設計原則やテスト駆動開発に沿って作業させることを目的としています。README によると、@poteto 氏のルール群（poteto-mode / pstack）の一部を土台にして内容を絞り込み、さらに決定的に結果が出る品質ゲートツールと組み合わせて構成されています。リポジトリには README.md と SKILL.md の2ファイルしかなく、プログラムではなく、エージェントへの指示文書として提供されている点を押さえておくとよいでしょう。

主な原則の構成

原則は三段階に分かれています。1つ目の First Make it Work では、Laziness Protocol、Foundational Thinking、TDD、Redesign From First Principles、Attack the Premise、Subtract Before You Add、Outcome-Oriented Execution、Build the Lever、そして Architecture（Model the Domain、Boundary Discipline、Type System Discipline、Make Operations Idempotent、Separate Before Serializing Shared State）が挙げられています。2つ目の Then Make it Right では、Fix Root Causes、Sequence Work into Verifiable Units、Test Behavior (Not Implementation)、各種ツールによる品質基準、Refactoring が並びます。3つ目の Practice では、Keep a Log、Guard the Context Window、Worktree Cleanup、小さく順序立てたコミットとプッシュが求められます。各原則の具体的な文言は SKILL.md に書かれていると考えられますが、README からは詳細は不明です。

品質ゲートツールとの連携

ub-mode の大きな特徴は、品質の判断をエージェントの主観に任せず、決定的なツールの数値で判定する点です。unclebob/crapper は関数ごとに循環的複雑度、テストカバレッジ、CRAP スコアを計算し、すべての関数が CRAP < 10 に収まることを求めます。unclebob/dryer は構造的なコード重複を検出し、類似度 0.8 以上の重複ブロックをなくすよう促します。unclebob/mutator はミューテーションテストを実行し、カバーされていないコードやテストで検出できない生存ミュータントを洗い出します。README では、カバーされていない箇所をテストで埋める、生存ミュータントを潰す、デッドコードを削除する、という対応が示されています。さらに unclebob/uml-viewer では、コンポーネント、カバレッジ、CRAP、ミューテーションスコア、依存方向を可視化できます。

使い方・導入方法

導入はとても簡単です。SKILL.md を、使っているエージェントのスキルディレクトリに ub-mode/SKILL.md として置きます。例として ~/.grok/skills/ub-mode/SKILL.md、~/.claude/skills/ub-mode/SKILL.md、~/.cursor/skills/ub-mode/SKILL.md が挙げられています。配置したら、エージェント上で /ub-mode と入力して呼び出します。ただし、品質ゲートを実際に機能させるには、crapper、dryer、mutator といった姉妹ツールも別途用意する必要があると考えられます。それらのインストール方法や、どの言語に対応しているかは、この README からは不明です。各ツールのリポジトリで確認してください。

注目ポイント・注意点

Clean Code などで知られる unclebob のアカウントで公開されており、多くのスターを集めています。テストや設計の規律を言葉で指示するだけでなく、CRAP スコアや重複度、ミューテーションテストといった客観的な指標と組み合わせている点が新しいところです。AIが生成するコードの品質を数値で管理したいチームにとって、参考になる設計といえます。一方で、ライセンスは明記されておらず、利用や改変の条件は README からは不明です。また、実体は Markdown のスキル文書なので、効果はエージェントがどこまで指示に従うかに左右されます。対応言語やツールの実行環境も README からは不明です。

技術構成: Markdown（SKILL.md によるエージェントスキル定義）、unclebob/crapper、unclebob/dryer、unclebob/mutator、unclebob/uml-viewer、Claude / Cursor / Grok などのエージェントのスキル機構
向いている人: Claude Code、Cursor、Grok などのAIコーディングエージェントを日常的に使っていて、生成されるコードの品質（テスト駆動、低複雑度、重複排除、ミューテーションテストによる検証）を規律として担保したい開発者やチームに向いています。AIに任せるとコードの質が落ちることに悩んでいる人や、ソフトウェア工学の原則をエージェント運用に落とし込む実例を学びたいエンジニア・学生にも参考になります。

総合 7. rddmwmddr/android-ad-cleaner

https://github.com/rddmwmddr/android-ad-cleaner

★ 4,902 ／ フォーク 440 ／ 言語 Python ／ 作成 2026-10-24 ／ MIT

中国メーカー製Androidスマホのシステム広告・ホット検索・ニュース推薦・営銷通知を、root不要かつ元に戻せる形でオフにするツール群

概要

OPPO/ColorOS、小米/HyperOS、vivo/OriginOS、荣耀/MagicOSなど中国国内向けAndroid端末に組み込まれた広告や推薦、マーケティング通知を、adb経由でシステムやアプリ標準の設定スイッチを操作して止めるプロジェクトです。rootや解锁、刷机は不要で、システム更新にも影響しない点を重視しています。使い方はAIエージェントに実行させる方法、チャットAIに手順を案内させる方法、手動チェックリスト、Mac/Windows向けデスクトップクライアントの4通りがあります。機種ごとの「機型脚本」で設定画面を自動操作し、通知はチャネル単位で営銷系だけを切る細やかさが特徴です。

要点

- root不要・アンロック不要で、変更は基本的に元に戻せる
- pm uninstallやdisable-userは使わず、標準の設定スイッチのみ操作
- uiautomator2で画面を読み、機種別スクリプトで自動操作
- AIエージェント用のAGENTS.mdを用意し一括実行に対応
- 営銷通知はチャネル単位で停止し、注文・配送通知などは残す

何をするものか

中国メーカー製Androidスマホのシステムに組み込まれた個性化広告、広告トラッキング(OAID)、ドロップダウン検索のホット検索、負一屏のニュース、ブラウザの情報フロー、アプリストアのカルーセルや起動広告、各アプリの「内容与营销」「活动优惠」通知などをオフにします。一方で検索機能、負一屏カード、ストアのインストール・更新、天気警報、宅配通知は残す方針です。広告コンポーネント(com.opos.ads、com.miui.systemAdSolutionなど)はpm suspendで一時停止しますが、これは端末所有者の同意が必要で、1コマンドで還元できます。快应用(インストール不要のミニアプリ)の検出と無効化にも対応しています。

主な機能と使い方の選択肢

利用方法は4つあります。1つ目はClaude Code、Codex、Cursor、Trae、WorkBuddyなどコマンドを実行できるAIエージェントにリポジトリをクローンさせ、AGENTS.mdを読ませて実行する方法。2つ目はDeepSeek、豆包、ChatGPTなどのチャットAIに提示語を渡し、スマホ単体で手順を案内してもらう方法。3つ目はブランド別の手動チェックリストに従う方法。4つ目はMac(Apple芯片/Intel)とWindows 10/11向けのデスクトップクライアントで、接続、検出、選択、実行、完了の流れをGUIで操作でき、PythonやAIツールは不要です。クライアントには診断情報付きでGitHub Issueを送るフィードバック機能もあります。

仕組み・アーキテクチャ

総入口はtool/run.pyで、devices、check(読み取り専用チェック)、backup(変更前スナップショット)、recipe(機型脚本の実行)、chan-scan/chan-off(通知チャネル)、quickapp、finish、dev-offといったサブコマンドを持ちます。tool/fast.pyがuiautomator2で画面を読み、入口を探してスイッチを切り替えるエンジンです。recipes/<ブランド>-<型番>/recipe.pyに機種ごとの「どのアプリを開き、どの文字をタップし、どのスイッチを切るか」が記述されています。システムのバージョンが合わない場合は警告し、入口が見つからない手順はスキップして誤タップを避ける設計です。新機種探索用のexplore.py(実験的)もあります。

技術構成

主要言語はPythonで、requirements.txtの依存はuiautomator2==3.7.1のみです。端末との通信はadbを前提とし、HDCを使う原生鸿蒙(HarmonyOS NEXT)は未対応です。デスクトップクライアントはpywebviewによる画面(gui/web/index.html)とPyInstallerでのパッケージングで構成され、Windowsインストーラ用のinstaller.iss、Mac用のbuild_mac.shがあります。アプリ名取得用にJavaで書かれたapplabels.dexも含まれます。testsディレクトリには実機を接続しないエンジンの回帰テストが多数あり、GitHub Actionsでテストとwindowsビルドのワークフローが用意されています。

導入方法

コマンドで使う場合は、まずスマホで開発者モードとUSBデバッグを有効化します(docs/00に画像付き手順あり)。その後 pip3 install -r requirements.txt を実行し、python3 tool/run.py devices でシリアル番号と機型脚本の有無を確認、-s <序列号> を付けてcheck、backup、recipe、chan-scan、chan-off、quickapp、finishの順に進め、最後にdev-off --yesで開発者モードを閉じます。GUIを使う場合はReleasesからdmg、setup.exe、portable.zipを入手します。Mac版は署名・公証済み、Windows版はSmartScreenの警告が出る場合があると説明されています。還元方法はdocs/05-還原.mdにまとめられています。

注目ポイント・注意点

実測済み機種は荣耀、一加、OPPO、小米、Redmi、vivo(iQOO含む)の二十数機種で、機種ごとに検証状況(実測通过、待复测など)が表で正直に示されています。同じブランドでもシステムの小バージョンで設定画面の名称や階層が変わるため、表にない機種はdocs/04の手順で脚本を書いてPRを送る形です。アプリのユーザー規約に代理で同意しない、アカウントや電話番号の画面をスクリーンショットしない、system分区を改変しないなど安全面の方針も明記されています。旧版は自動更新されません。利用による問題は自己責任と声明されています。ライセンスはMITです。

技術構成: Python、uiautomator2、adb、pywebview、PyInstaller、Java(applabels.dex)、Inno Setup(installer.iss)、GitHub Actions
向いている人: OPPO、小米、vivo、荣耀、一加など中国国内版Android端末を使っていて、システム広告や推薦通知を安全かつ可逆的に減らしたい人。root化を避けたい一般ユーザーはデスクトップクライアント、AIエージェントによる端末自動操作やuiautomator2によるUI自動化に興味のあるエンジニア・学生には、ソースや機種別スクリプトの設計が参考になります。

総合 8. JustAResearcher/gpu-unlock-ada

https://github.com/JustAResearcher/gpu-unlock-ada

★ 4,546 ／ フォーク 580 ／ 言語 C ／ 作成 2026-10-23 ／ GPL-2.0

RTX 4090などAda世代GPUのFP32累積テンソル演算性能を約2倍に引き上げるLinuxカーネルモジュール

概要

このリポジトリは、標準のNVIDIAドライバが起動する前にGPUのSM speed-select設定を書き換え、FP16/BF16のFP32累積テンソル演算スループットを解放するLinuxカーネルモジュールです。RTX 4090では命令レベルで約165 TF/sから約331 TF/sへの向上が報告されており、RTX 4070 Ti SUPERなど他のAda世代GPUも実験的に対応しています。改変版NVIDIAドライバやUEFIアプリは不要で、署名済みファームウェアと公開済みペイロードを使い、FLR後に無改変のドライバへ引き渡す点が特徴です。ファームウェアはユーザーのマシン上でNVIDIA公式配布物から生成する方式をとっており、バイナリは配布していません。

要点

- RTX 4090でFP16/BF16→FP32累積の命令スループットが約2倍
- INT8・dp4a・スカラーFP32はほぼ変化なし
- 実アプリでの効果は限定的で、cuBLAS GEMMでは形状により0〜34%程度
- 他のAda GPUは個別のopt-inフラグとPCIアドレス指定が必要な実験扱い
- 初回はコールドブート必須、NVIDIAファームウェアはローカル生成

何をするものか

Linux起動時、NVIDIAドライバがGPUを掴む前にカーネルモジュールgpu_unlockが動作し、GPUのSM speed-select（SS0とSS1）を上書きします。これにより、GeForceで制限されているFP32累積のテンソル演算レートが解除されます。RTX 4090（10de:2684, AD102）では、FP16→FP32累積が165.6から331.6 TF/s、BF16→FP32累積が165.5から330.6 TF/sに向上したと報告されています。設定はPCIe FLR（Function Level Reset）やNVIDIA 615.78.08ドライバの初期化を経ても維持されたとされています。元はZagitta/gpu-unlock（Ampere向け）のフォークで、TOTORONG/AD103_win_unlockerのAD102ペイロードを組み込んでいます。

仕組み・アーキテクチャ

RTX 4090向けの処理では、まず署名済みファームウェアを実行し、ペイロードの完了マーカーと必要な権限を確認します。その後SS0に0x88888888、SS1に0x8を書き込み、FLRを行ってから無改変のNVIDIAドライバにバインドさせます。FLRの前後で、期待どおりの実効speed-select値が読み出せることも確認します。ペイロードは公開済みのdebug-booter解析から導いた命令アドレスを使用し、本番用booterはGPU内部で認証・復号されます。カーネル側はロードする各アセットをpin留めしたSHA-256で検証してから、カードの確保やファームウェアDMAを始めます。WPR2が既に初期化されているカードや、既にバインド済み・電源断状態のカードは処理をスキップします。

対応ハードウェア

RTX 4090はAD102用のファームウェアとペイロードが自動で選ばれます。従来のGA102/GA104（GeForce/CMP）とGA103/GA106のデバイスリストも残っており、GA107のRTX 3050 6GBも1枚で成功例があります。実験的なAda対応には、RTX 4090 D、RTX 4080/4080 SUPER、RTX 4070系、RTX 4060系、ノートPC向けRTX 4090/4060/4050 Laptopなどが含まれます。いずれもIDごとに専用のフラグ（ad103_experimentalなど）が必要で、多くは正確なPCIアドレスの指定も求められ、targets=allでは選択されません。検証は各IDにつき1枚から数枚程度です。ノートPCはVBIOSやEC、FLRの扱いが機種ごとに異なり、結果は他機種に流用できないと明記されています。

使い方・導入方法

git cloneした後にmake installを実行すると、ディストリビューションを判別してパッケージをビルドしインストールします。明示的なターゲットとしてmake install-arch、install-deb、install-rpmもあります。ビルド時にはpin留めされたNVIDIA 610.43.03の入力をHTTPSで取得し、SHA-256で検証してファームウェアを生成します。カーネルヘッダとDKMSが必要です。モジュールはinitramfs（mkinitcpio、initramfs-tools、dracut）に組み込まれ、カーネルコマンドラインにgpu_unlock.targets=0000:03:00.0のような指定をしたときだけ有効になります。成否はjournalctl -k -b | grep gpu_unlockでgpu_unlock: unlockedの表示を見て確認します。無効化するには、コマンドラインからtargets指定を外します。

技術構成と検証資産

本体はC言語のLinuxカーネルモジュール（kernel/gpu_unlock）です。Debian、RPM、Arch向けのパッケージングとDKMS設定が含まれます。diagnostics/benchにはCUDAによる命令ベンチマーク（mma_bench.cu、tensor_modes_issue.cu）やGEMM検証、diagnostics/peekには読み取り専用のBAR0サンプラーがあります。make testはGPUにアクセスしないホスト側テストで、make test-fullは生成アセットを必須とする厳格版です。いずれもファームウェア実行をモックしたものです。docs/evidence以下には、カードやドライバのバージョンごとに、stockとunlockedを対比した計測ログとSHA256SUMSが大量に収録されています。Windows向けのAD104移植はリファレンスとして別扱いになっています。

注目ポイント・注意点

README自身が、数値はレジスタ常駐の命令発行レートであり、アプリ全体の高速化ではないと何度も強調しています。量子化MoEの推論では効果がなく（0.983倍）、FP8 GEMMも計測条件によって差が出ませんでした。電力上限が固定されている場合、unlocked状態の方が消費電力が上がり、SW Power Capにかかる例も報告されています。初回は必ず電源を完全に切ってからのコールドブートで試す必要があります。NVIDIAドライバのロード後に手動でモジュールを入れても解除できません。ファームウェアが停止しなかった場合は強制アンロードせず電源断するよう指示されています。長期の安定性テストは限定的で、保証や法的な扱いについてはREADMEからは不明です。ライセンスはGPL-2.0-onlyで、NVIDIAファームウェアには別の条件が適用されます。

技術構成: C、Linux kernel module、DKMS、CUDA、Python、Makefile、mkinitcpio / initramfs-tools / dracut、Debian packaging / RPM / makepkg、cuBLAS / CUTLASS（ベンチマーク）
向いている人: RTX 4090などAda世代GeForceを所有し、Linux上でFP16/BF16のFP32累積テンソル演算（学習や非量子化GEMMなど）の性能を引き出したい上級ユーザーや研究者に向いています。GPUファームウェアやカーネルモ

ジュールのリバースエンジニアリングに関心がある人にとっては、豊富な検証記録も参考になります。一方で、コールドブートの手順やカーネルコマンドラインの編集、障害時の切り戻しを自分で扱える知識が前提となります。実験的なハードウェア改変のリスクを理解したうえで使う必要があります。

総合 9. interplains/Zero-Breach-Toolkit

https://github.com/interplains/Zero-Breach-Toolkit

★ 3,701 ／ フォーク 393 ／ 言語 Python ／ 作成 2026-10-25 ／ Apache-2.0

Pegasus や Predator など既知の傭兵型スパイウェアの痕跡を、PC やスマホのデータからオフラインで調べる無料のPython製ツール

概要

Zero Breach Toolkit は、Pegasus、Predator、Graphite、Candiru といった商用(傭兵型)スパイウェアの公開済み痕跡(IOC)を探すツールです。Windows・macOS・Linux のPCに加え、AndroidQF で取得した Android の収集データや iPhone のローカルバックアップも検査できます。読み取り専用で、スキャン中はネットワークにアクセスしません。報告書では、何が見つかったか、その証拠がどれだけ強いか、何を調べられなかったかを必ず明示します。技術知識のないジャーナリストや活動家でも使えるよう、手順が丁寧に書かれている点も特徴です。

要点

- 既知スパイウェアの公開IOC 約12,400件(29ソース)と照合する
- 読み取り専用で、スキャン中は Python のネットワーク呼び出しを遮断する
- PCとAndroid収集データ、iPhoneバックアップに対応
- 「検出なし」を安全の証明として扱わず、調べられなかった範囲を必ず報告する
- 追加パッケージ不要、Python 3.8以上だけで動き、ビルドは再現可能

何をするものか

国家レベルの標的型攻撃で使われる商用スパイウェア(Pegasus、Predator、Graphite、Candiru など)について、公開済みの痕跡を端末から探すトリアージツールです。照合に使うのは、Amnesty International の Security Lab、Citizen Lab、Mobile Verification Toolkit(MVT)プロジェクト、Echap が公開している約12,400件の指標です。対象はPCのブラウザ履歴やシステム記録、Android の収集データ、iPhone のバックアップです。README は最初に「今まさに狙われていると思うなら、まず専門家に相談を」と案内し、Access Now の Digital Security Helpline などを紹介しています。あくまで一次確認のための道具という位置づけです。

主な機能

Windows では次のような項目を調べます。いずれも管理者権限は不要です。 ・Chrome、Edge、Firefox、Tor Browser など多数のブラウザの履歴、Cookie、HSTS、キャッシュ ・Chromium を内蔵するデスクトップアプリ ・ダウンロードファイルの Mark of the Web ・Win+R の入力履歴、PowerShell ログ、Defender の履歴 ・Prefetch、Amcache、DNS キャッシュ、スタートアップ登録 ・指定フォルダ内ファイルのハッシュ macOS と Linux では、Safari の各種データベース、LaunchAgent、systemd、cron、シェル履歴などを調べます。Android では、インストール履歴、署名証明書、SMS 内のリンク、改造端末の兆候を確認します。iPhone では、メッセージ、Safari、診断ログなど、iOS 15〜18 のデータ構造に対応しています。

仕組み・アーキテクチャ

コードは scan/ 以下にまとまっています。 ・入口:scan/zero-breach-toolkit.py ・本体:scan/zbtriage/ パッケージ(指標の読み込み、ネットワーク遮断を担う netguard、レポート生成など) ・検査ごとの処理:scan/zbtriage/modules/ に browser、android、ios、hashscan、posix などが並ぶ 各リーダーは1つのレジストリを共有しており、プラットフォーム選択、カバレッジ説明、単一ファイルビルドに使われます。iPhone の SQLite WAL は一時コピー上で検証・再生するため、元データは変更されません。Android 端末とは直接通信せず、AndroidQF の収集結果を読みます。レポートでは各検出に証拠グレードが付き、「Worth a look(要確認)」「時系列」「確認できなかった範囲」に分けて示されます。

使い方・導入方法

Python 3.8 以上を入れ、ZIP をダウンロードするか git clone します。 ・指標リストの更新:python -X utf8 scan/zero-breach-toolkit.py --update(インターネットを使うのはこの手順だけ) ・検査の実行:オプションなしで起動するとメニューが出て、Enter で標準検査が始まる(約15分) ・Windows では scan/run-scan.cmd をダブルクリックしても実行できる 主なオプションは次のとおりです。 ・--quick:高速版 ・--phone "FOLDER":スマホの収集データやバックアップを検査 ・--json:機械可読なレポート ・--share-json:共有用の最小サマリ ・--scan-type:検査範囲を full、targeted、browser から選ぶ 暗号化された iPhone バックアップには、別途 requirements-backups.txt から cryptography パッケージを入れる必要があります。macOS では、ターミナルにフルディスクアクセスを与えておかないと Safari を検査できません。

技術構成と検証性

主要言語は Python で、標準ライブラリだけで動きます。暗号化バックアップの解析だけは、オプションの cryptography(バージョンとハッシュを固定)が必要です。 ・Candiru 検出には YARA ルールを同梱(third_party/candiru) ・dist/ には STIX、MISP、hosts 形式などの指標エクスポートがある ・build/bundle.py が配布用の単一ファイルを決定論的にビルドする。SHA-256 を照合すれば、リリース物がソースと一致することを自分で確かめられる ・リリースには GPG 署名が付く ・GitHub Actions でテストとリリース検証を実行している ・AI アシスタント向けに --describe や SKILL.md も用意されている ライセンスは、コードが Apache-2.0、キュレーション済み指標が CC0 です。

注目ポイント・注意点

見つけられるのは公開済みの指標だけです。新しいスパイウェアや、標的ごとに作り直された検体は検出できません。痕跡の多くも数週間で消えます。「一致あり」は「感染している」ではなく「詳しく見るべき」という意味で、専門家によるフォレンジックの代わりにはなりません。レポートにはユーザー名や閲覧履歴など個人情報が含まれるので、共有の扱いには注意が必要です。プロジェクトは匿名運営で、署名付きの Windows 実行ファイルは提供しない方針です。README はソースから実行し、自分で検証することを勧めています。なお、メタデータ上の作成日時が 2026年 となっている点は、README からは理由が不明です。

技術構成: Python 3.8+、SQLite(WAL解析)、cryptography(オプション)、YARA ルール、AndroidQF、STIX / MISP、GitHub Actions
向いている人: スパイウェアの標的になりうるジャーナリスト、活動家、人権擁護者が最初に行う確認に向いていま

す。そうした人を支援するデジタルセキュリティ担当者や、IOC ベースのフォレンジック・トリアージ手法を学びたいエンジニアや学生にも参考になります。

総合 10. anhao/ai-pose-gallery-zh

https://github.com/anhao/ai-pose-gallery-zh

★ 3,612 ／ フォーク 934 ／ 言語 HTML ／ 作成 2026-10-27 ／ NOASSERTION

日本語のAIポーズ参考画像サイトを全量ミラーし、中国語化した上で依存ゼロの静的ギャラリーとして公開するリポジトリ

概要

原作者ギア氏が公開しているAI絵画・3D・撮影向けの人体ポーズ参考図鑑は、メタデータが日本語のみで一括ダウンロードもできませんでした。本リポジトリは1439件分のサムネイルと原寸画像をすべて取得し、体系・分類・ポーズ名・視角を中国語に翻訳したうえで、フレームワーク不要のレスポンシブなギャラリーとしてCloudflare Pagesにそのままデプロイできる形にまとめています。さらにClaude Codeなどで使えるAgent用のSKILL.mdも同梱しており、画像生成や動画生成時のポーズ参照素材として使えるようにしている点が特徴です。

要点

- 288種のポーズ・1439枚の多角度参考画像を全量ミラー
- 日本語メタデータを323語の対訳辞書で完全中国語化
- 依存ゼロのindex.html単体でフィルタ・灯箱・DLを実装
- Cloudflare Pages向けビルドスクリプトと_headersを同梱
- Claude Code等で使えるpose-reference系Skillsを5種提供

何をするものか

日本の原サイト ai-pose-gallery.pages.dev で公開されている、AI絵画や3D、撮影の参考用人体ポーズ画像集を中国語で使えるようにしたミラーです。README によると、全身単人・標準系365件、全身単人・セクシー系399件、上半身単人400件、二人系275件の計1439件があり、各ポーズは正面・斜め・側面・俯瞰・煽りの5視角のうちいくつかを持ちます。なおリポジトリ説明文では「1439组 / 2878张」、README本文では「288组 / 1439张」と表記が異なりますが、README の内訳ではポーズ名288種、レコード1439件、サムネイルと原寸画像を合わせて2878ファイルと読めます。成人向けの分類も含まれる点に注意が必要です。

主な機能

ギャラリーは4つの体系タブを持ち、それぞれ画像数とポーズ組数を表示します。ポーズ名・番号・ID・分類キーワードで検索でき、スラッシュキーで検索欄にフォーカスできます。動作分類と5種の視角を組み合わせたフィルタ、ポーズ単位のグループ表示と1枚ずつの表示の切り替え、サムネイルのホバーやクリックによる角度切り替えに対応します。ライトボックスではまずサムネイルを表示してから900×1200の原寸PNGを読み込み、PNGの直接ダウンロード、IDコピー、新規ウィンドウ表示が可能です。矢印キーで角度とポーズを移動、Escで閉じ、スマホではスワイプ操作もできます。

仕組み・アーキテクチャ

構成は単一の index.html と JSON データ、画像フォルダだけの純粋な静的サイトです。フロントエンドは fetch で data/pose-zh.json を読み込み、検索用に連結した「检索」フィールドを使ってあいまい検索を行います。元の日本語データ data/pose.json は改変せずに保持され、翻訳は scripts/i18n.py の辞書（重複除去後323件）を scripts/apply_translation.py で流し込んで中国語版を生成する流れです。画像取得には48スレッド並列・レジューム・自動リトライ対応の download.py、ファイルの存在・サイズ・画像ヘッダを確認する verify.py が用意されています。SEO面では canonical、Open Graph、Twitter Card、JSON-LD（WebSite と ImageGallery）、robots.txt、sitemap.xml まで整備されています。

使い方・導入方法

JSONを fetch で読むため、index.html をダブルクリックで開くとCORSで弾かれます。ローカルでは python3 -m http.server 8000 を実行して http://127.0.0.1:8000 を開きます。Cloudflare Pagesへは、Git連携で Framework preset を None、Build command を bash scripts/build.sh、出力ディレクトリを dist に設定すれば push ごとに自動公開されます。コマンドラインからは bash scripts/build.sh の後に npx wrangler pages deploy dist --project-name ai-pose-gallery-zh で直接アップロードできます。出力先をルートにすると .git 内のpackファイルが25MiB制限を超えて失敗するため、dist の利用が推奨されています。Fork して別ドメインで使う場合は index.html や robots.txt、sitemap.xml 内のURLを置き換える必要があります。

Agent向けSkills

skills/ 配下には SKILL.md 形式のスキルが5種あり、pose-reference-standard、pose-reference-upper、pose-reference-sexy、pose-reference-couple、そして全件を対象とする pose-reference-all が含まれます。各スキルは SKILL.md と references/catalog.csv を持ち、csv の画像列は ../../img/<id>.png という相対パスでリポジトリの img/ を再利用します。そのためリポジトリ全体をcloneしたまま、スキルディレクトリを ~/.claude/skills/ にシンボリックリンクで配置する必要があり、コピーすると画像が参照できません。画像を同梱した単体配布版としては別リポジトリ jasoncai0/pose-reference-skills が紹介されています。

注目ポイント・注意点

ライセンス表記はNOASSERTIONで、README によれば scripts/ と index.html のコードのみがMITライセンスです。img/、thumbs/、data/ の画像と元データの著作権は原作者ギア氏にあり、MITの対象外と明記されています。リポジトリは個人の学習・翻訳・静的サイト実践のアーカイブという位置付けで、商用利用の許諾にはならず、権利者から申し立てがあれば削除するとしています。また成人向けのポーズや性的な用語を含む分類があり、未成年の閲覧は不可とされています。画像総量は約659.5MBと大きいため、clone時の容量にも注意が必要です。日本語話者にとっては、日中対訳表 translations.csv が辞書的な資料としても参考になるでしょう。

技術構成: HTML、JavaScript（fetchによるJSON読み込み）、JSON、Python（download.py、i18n.py、apply_translation.py、verify.py）、Bash（build.sh）、Cloudflare Pages、wrangler、SKILL.md（Claude Code等のAgentスキル）
向いている人: AI画像生成・3D・イラスト・撮影でポーズ参考資料を探しているクリエイター、Claude Code などのAgentに参照用ポーズ画像を渡したい人、依存ゼロの静的ギャラリーやCloudflare Pagesへのデプロイ、SEO設定、

多言語化の実例を学びたいフロントエンド学習者に向いています。ただし成人向けコンテンツを含み、画像の権利は原作者に帰属するため、利用目的には注意が必要です。